

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	Shaik Basheer	Reg. No.:	192465049
Section / Batch:	Slot-B-AIDS/2024-28	Date:	26-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NPComplete.	Q1

Code of Conduct:

I Shaik Basheer / 192465049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Shaik Basheer

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Boolean Circuit Satisfiability (NP-Complete Problem)

Determine whether a Boolean circuit produces a true output for some input assignment. Clearly define

constraints related to logical gate operations. Analyse how constraints propagate through the circuit.

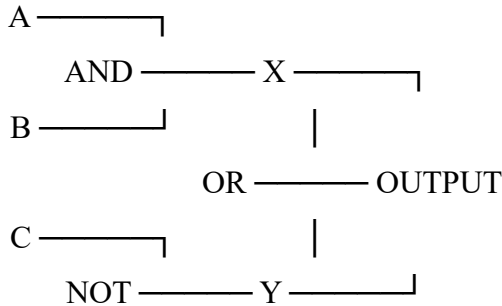
Develop a backtracking-based solution to test input combinations. Apply pruning techniques to eliminate

invalid assignments. Justify correctness and explain NP-Complete nature.

1. Introduction

Boolean Circuit Satisfiability is a constraint-based problem in which a Boolean circuit is given as a combination of logical gates such as AND, OR, and NOT, together with Boolean input variables. The objective is to determine whether there exists at least one assignment of 0 and 1 to the input variables that makes the final output of the circuit equal to 1.

For example, consider a circuit:



The circuit produces:

$X = A \text{ AND } B$

$Y = \text{NOT } C$

$\text{OUTPUT} = X \text{ OR } Y$

The problem is to determine whether there is an assignment of A, B, and C such that:

$\text{OUTPUT} = 1$

Boolean Circuit Satisfiability is an important computational problem because it demonstrates constraint satisfaction, backtracking, pruning, and complexity classification.

The problem belongs to the NP-Complete class. A proposed input assignment can be verified by evaluating the circuit in polynomial time, while finding a satisfying assignment may require exploring exponentially many possibilities.

2. Input Conditions

The input consists of:

A Boolean circuit.

A set of Boolean input variables.

Logical gates such as AND, OR, and NOT.

Connections between gates.

One final output gate.

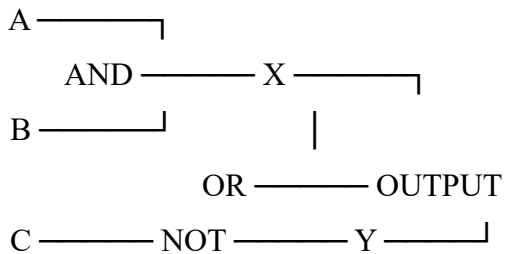
Each input variable can take only one of two values:

0 = False

1 = True

Example Input Circuit

Consider:



The corresponding Boolean expressions are:

$$X = A \text{ AND } B$$

$$Y = \text{NOT } C$$

$$\text{OUTPUT} = X \text{ OR } Y$$

The objective is:

Determine whether OUTPUT can become 1.

3. Constraints

Each logical gate imposes a specific constraint on its inputs and output.

AND Gate

For an AND gate:

$$X = A \text{ AND } B$$

The output is 1 only when both inputs are 1.

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

Therefore:

$$X = 1 \rightarrow A = 1 \text{ AND } B = 1$$

OR Gate

For an OR gate:

$$X = A \text{ OR } B$$

The output is 1 when at least one input is 1.

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

Therefore:

$$X = 0 \rightarrow A = 0 \text{ AND } B = 0$$

NOT Gate

For a NOT gate:

$$X = \text{NOT } A$$

The output is the opposite of the input.

A	X
0	1
1	0

Therefore:

$$X = 1 \rightarrow A = 0$$

$$X = 0 \rightarrow A = 1$$

4. Interaction of Constraints

The constraints of different gates interact with each other because the output of one gate can become the input of another gate.

Consider:

$$X = A \text{ AND } B$$

$$Y = \text{NOT } C$$

$$\text{OUTPUT} = X \text{ OR } Y$$

Suppose we require:

$$\text{OUTPUT} = 1$$

Since:

$$\text{OUTPUT} = X \text{ OR } Y$$

at least one of X or Y must be 1.

Therefore:

$$X = 1 \text{ OR } Y = 1$$

If:

$$X = 1$$

then:

$$A = 1$$

$$B = 1$$

If:

$$Y = 1$$

then:

$$C = 0$$

Thus, a constraint at the final output propagates backwards through the circuit.

This is called constraint propagation because a restriction on one gate can reduce the possible values of variables connected to that gate.

Example

Suppose:

$$\text{OUTPUT} = 0$$

For:

$$\text{OUTPUT} = X \text{ OR } Y$$

both inputs must be zero:

$$X = 0$$

$$Y = 0$$

Since:

$$X = A \text{ AND } B$$

$X = 0$ means at least one of A or B must be 0.

Since:

$$Y = \text{NOT } C$$

and:

$$Y = 0$$

we obtain:

$$C = 1$$

Therefore, the final output constraint has reduced the possible input assignments.

5. Backtracking Approach

Backtracking constructs a Boolean assignment incrementally.

For n input variables, each variable has two possible values:

0 or 1

The basic procedure is:

- Select an unassigned input variable.
- Assign 0.
- Propagate the constraints through the circuit.
- Check whether the partial assignment can still produce output 1.
- If it is invalid, reject the branch.
- Otherwise continue with the next variable.
- If necessary, backtrack and assign 1.
- Continue until a satisfying assignment is found.
- If all assignments fail, report that the circuit is unsatisfiable.

6. State-Space Representation

For n Boolean variables, the complete search space contains:

2^n

possible assignments.

For example, if there are three variables:

A, B, C

the search space is:

000

001

010

011

100

101

110

111

A state-space tree can be represented as:

```

      {}
    /  \
  A=0   A=1
 /  \   /  \
B=0 B=1 B=0 B=1
|   |   |   |
... ..

```

Backtracking does not necessarily visit all 2^n combinations because invalid branches can be removed early.

7. Basic Algorithm

BooleanCircuitSAT()

BooleanCircuitSAT(variables):

 If all variables are assigned:

 Evaluate the circuit

 If output == 1:

 return TRUE

 Else:

 return FALSE

 Select an unassigned variable

 Assign variable = 0

 If partial assignment is consistent:

 If BooleanCircuitSAT(variables):

 return TRUE

 Assign variable = 1

 If partial assignment is consistent:

 If BooleanCircuitSAT(variables):

 return TRUE

 Unassign variable

 return FALSE

The important part is the consistency check.

Before continuing with a branch, the algorithm determines whether the current partial assignment can still satisfy the circuit.

8. C++ Implementation

```
main.cpp
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  bool canSatisfy(vector<int>& a)
5  {
6      int A = a[0];
7      int B = a[1];
8      int C = a[2];
9
10     // X = A AND B
11     int X;
12
13     if (A == -1 || B == -1)
14     |   X = -1;
15     else
16     |   X = A & B;
17
18     // Y = NOT C
19     int Y;
20
Satisfying Assignment Found:
A = 0
B = 0
C = 0
Circuit Output = 1

...Program finished with exit code 0
Press ENTER to exit console.
```

Note:

The circuit used in the program is:

$X = A \text{ AND } B$

$Y = \text{NOT } C$

$\text{OUTPUT} = X \text{ OR } Y$

One satisfying assignment is:

$A = 0$

$B = 0$

$C = 0$

because:

$X = 0 \text{ AND } 0 = 0$

$Y = \text{NOT } 0 = 1$

$\text{OUTPUT} = 0 \text{ OR } 1$

$= 1$

Therefore, the circuit is satisfiable.

9. Constraint Propagation and Pruning

Constraint propagation reduces the search space before all variables are assigned.

1. Output-Based Propagation

If:

$$\text{OUTPUT} = X \text{ OR } Y$$

and the required output is:

$$\text{OUTPUT} = 0$$

then:

$$X = 0$$

$$Y = 0$$

This immediately restricts the circuit.

2. AND Gate Pruning

For:

$$X = A \text{ AND } B$$

if:

$$A = 0$$

then:

$$X = 0$$

regardless of the value of B.

Therefore, there is no need to explore the other possibility of B to determine X.

3. OR Gate Pruning

For:

$$X = A \text{ OR } B$$

If:

$$A = 1$$

then:

$$X = 1$$

regardless of B.

Therefore, the value of B does not need to be explored for determining this gate's output.

4. NOT Gate Propagation

For:

$$X = \text{NOT } A$$

if:

$$X = 1$$

then:

$$A = 0$$

This directly determines the input.

5. Early Pruning

A branch is abandoned immediately when the partial assignment proves that the final output cannot become 1.

For example:

OUTPUT = X AND Y

If:

$X = 0$

then:

OUTPUT = 0

regardless of Y.

Therefore, the branch can immediately be rejected.

This prevents unnecessary exploration of invalid assignments.

10. Correctness Justification

The algorithm is correct because it systematically explores possible Boolean assignments while checking the constraints imposed by the circuit.

For every input variable, the algorithm considers both possible values:

0 and 1

Therefore, every possible Boolean assignment is either explored or eliminated only when constraint propagation proves that it cannot produce a satisfying output.

If the algorithm reaches a complete assignment and the circuit evaluates to:

OUTPUT = 1

then a valid satisfying assignment has been found.

If a partial assignment cannot satisfy the circuit, the corresponding branch is safely pruned because no extension of that assignment can produce a valid solution.

If one branch fails, the algorithm backtracks and tries the alternative assignment.

Therefore:

If the algorithm returns TRUE, the circuit is satisfiable.

If the algorithm explores all possible assignments and returns FALSE, no satisfying assignment exists.

Hence, the backtracking algorithm is correct.

11. Performance Analysis

Let:

n = number of Boolean input variables

Each variable has two possible values:

0 or 1

Therefore, the maximum number of assignments is:

2^n

The worst-case time complexity is approximately:

$O(2^n \times G)$

where:

G = number of gates in the circuit

because each complete assignment may require evaluating the circuit.

With efficient constraint propagation, many branches can be eliminated before reaching a complete assignment.

Therefore, the practical execution time can be significantly lower than the worst-case exponential search.

Space Complexity

The recursion depth is at most:

$O(n)$

for n input variables.

Additional memory is required for storing:

Input assignments

Gate values

Circuit structure

Recursion information

Thus, the basic auxiliary space is approximately:

$O(n + G)$

12. NP-Complete Classification

Boolean Circuit Satisfiability belongs to the NP-Complete class.

NP

A problem belongs to NP if a proposed solution can be verified in polynomial time.

For Boolean Circuit Satisfiability, suppose someone provides an input assignment:

$A = 1$

$B = 0$

$C = 1$

...

We can evaluate all gates in the circuit and determine the final output.

Circuit evaluation requires time proportional to the size of the circuit:

$O(G)$

Therefore, a proposed satisfying assignment can be verified in polynomial time.

Hence:

Circuit SAT $\in$ NP

NP-Hard

Boolean Circuit Satisfiability is also NP-Hard because general Boolean satisfiability problems can be represented as Boolean circuits, and known NP problems can be reduced to circuit satisfiability in polynomial time.

Therefore, Circuit SAT is at least as difficult as every problem in NP.

Thus:

Circuit SAT is NP-Hard

Combining both properties:

Circuit SAT $\in$ NP

Circuit SAT is NP-Hard

Therefore:

Circuit SAT is NP-Complete

13. Performance Challenges

The main challenge is the exponential number of possible input assignments.

For n variables:

Variables	Maximum Assignments
5	32
10	1,024
20	1,048,576
30	1,073,741,824
40	1,099,511,627,776

Therefore, directly testing every combination becomes expensive as the number of variables increases.

Performance can be improved using:

- Constraint propagation
- Early pruning
- Forward checking
- Variable ordering
- Most-constrained-variable selection
- Memoization
- Efficient circuit evaluation

These techniques reduce unnecessary branches in the search tree.

14. Example of Backtracking with Pruning

Consider:

OUTPUT = (A AND B) OR (NOT C)

Initially:

A = ?

B = ?

C = ?

Suppose the algorithm selects:

C = 0

Then:

NOT C = 1

Therefore:

OUTPUT = (A AND B) OR 1
= 1

The algorithm immediately knows that the circuit is satisfiable.

There is no need to explore all combinations of A and B.

This demonstrates the advantage of pruning and constraint propagation.

15. Conclusion

Boolean Circuit Satisfiability demonstrates how constraint-based problem solving and backtracking can be applied to an NP-Complete problem.

Each Boolean input has a domain of:

$\{0,1\}$

while AND, OR, and NOT gates impose logical constraints on the variables and intermediate outputs. These constraints propagate through the circuit and can be used to identify invalid assignments early.

The backtracking algorithm systematically explores possible input assignments and backtracks whenever a partial assignment cannot satisfy the circuit. Pruning techniques such as output-based reasoning, gate-level propagation, and early rejection significantly reduce the search space.

Although the worst-case search space is exponential, approximately:

$$O(2^n)$$

for n Boolean variables, constraint propagation and pruning can substantially improve practical performance.

Finally, Boolean Circuit Satisfiability is NP-Complete because a proposed assignment can be verified in polynomial time and the problem is NP-Hard through polynomial-time reductions from NP problems. Thus, it provides a strong example of the relationship between backtracking, constraint propagation, pruning, and computational complexity.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

